

Fitness & Wellness App Database

Fitness App Overview

The Fitness & Wellness App is designed to manage user health and fitness data. The app's system supports user account creation, personalized workout plans, tracked diet/fitness plans based on users' desires, interaction with trainers, and making payments. The database stores and organizes data efficiently so users can access health information and the company can manage services smoothly. Security features include hashed passwords, unique emails, safer payment storage, role-based access for users/trainers/admins, restricted views for sensitive health data, two-factor authentication support, backup code hashes, login attempt tracking, and audit logs for key record changes. The database for this app must support accurate recordkeeping, strong relationships between data, and proper constraints to maintain data integrity.

Requirements

1. Store user account information as a strong entity, including: UserID (PK, auto-increment), FullName (NOT NULL), Email (UNIQUE, NOT NULL), PasswordHash (NOT NULL), Age, Gender, PhoneNumber, and RegistrationDate (NOT NULL, defaults to current date). Passwords must be stored as hashes instead of plain text, and user emails must be unique to prevent duplicate accounts. Each user can be associated with subscriptions, workout plans, diet plans, progress records, payments, roles, and two-factor authentication settings.
2. Store trainer information as a strong entity, including: TrainerID (PK, auto-increment), FullName (NOT NULL), Email (UNIQUE, NOT NULL), Specialty, Certification, and PhoneNumber. Trainer emails must be unique to prevent duplicate trainer accounts. A trainer can be assigned to many workout plans and diet plans, but each plan belongs to only one trainer.
3. Store administrator account information, including: AdminID (PK, auto-increment), FullName (NOT NULL), Email (UNIQUE, NOT NULL), PasswordHash (NOT NULL), and CreatedDate (NOT NULL, defaults to current date). Admin accounts support system management and must store passwords as hashes.
4. Support role-based access control, including: RoleID (PK, auto-increment), RoleName (UNIQUE, NOT NULL), UserID (FK referencing Users, NOT NULL), and RoleID (FK referencing Roles, NOT NULL). Roles separate access levels for users, trainers, and administrators.
5. Store subscription information, including: SubscriptionID (PK, auto-increment), UserID (FK referencing Users, NOT NULL), PlanType (NOT NULL), StartDate (NOT NULL),

EndDate, and Status (e.g., Active, Expired, Canceled, Pending, NOT NULL). Each user can have one or more subscriptions, but each subscription belongs to exactly one user. Subscription changes must be tracked in the audit log.

6. Track payments made by users, including: PaymentID (PK, auto-increment), UserID (FK referencing Users, NOT NULL), SubscriptionID (FK referencing Subscriptions, optional), PaymentDate (NOT NULL, defaults to current date), Amount (NOT NULL, must be greater than 0), PaymentMethod (e.g., Credit Card, PayPal, Apple Pay, NOT NULL), PaymentStatus (e.g., Completed, Pending, Failed, Refunded, NOT NULL), TransactionID, PaymentProvider, and CardLastFour. Payment records must not store full credit card numbers, CVV codes, or full card details. Payment updates must be tracked in the audit log.
7. Store workout plans, including: WorkoutPlanID (PK, auto-increment), UserID (FK referencing Users, NOT NULL), TrainerID (FK referencing Trainers, NOT NULL), PlanName (NOT NULL), DifficultyLevel, DurationWeeks, and Goal. Each workout plan is assigned to exactly one trainer and one user, but a trainer can manage many workout plans and a user can have many workout plans. A workout plan can contain many exercises, and the same exercise can appear in multiple plans. This many-to-many relationship is resolved by WorkoutPlanExercises.
8. Track workout sessions as an entity dependent on WorkoutPlan, including: SessionID (PK, auto-increment), WorkoutPlanID (FK referencing WorkoutPlans, NOT NULL), SessionDate (NOT NULL), DurationMinutes, CaloriesBurned, and CompletionStatus (e.g., Completed, Skipped, In Progress). Each workout session belongs to exactly one workout plan. A separate UserID is not required because the related user can be identified through WorkoutPlans.
9. Store exercise information as a strong entity, including: ExerciseID (PK, auto-increment), ExerciseName (NOT NULL), Description, TargetMuscleGroup, and EstimatedCaloriesBurned. Exercises are reusable across many workout plans. The many-to-many relationship between Exercise and WorkoutPlan is managed through the WorkoutPlanExercises junction entity.
10. Store diet plans, including: DietPlanID (PK, auto-increment), UserID (FK referencing Users, NOT NULL), TrainerID (FK referencing Trainers, NOT NULL), PlanName (NOT NULL), CalorieTarget, and DurationDays. Each diet plan belongs to exactly one user and is managed by one trainer. A diet plan can include many meals, and each meal belongs to exactly one diet plan.
11. Store meal information as an entity dependent on DietPlan, including: MealID (PK, auto-increment), DietPlanID (FK referencing DietPlans, NOT NULL), MealName (NOT NULL), MealType (e.g., Breakfast, Lunch, Dinner, Snack), CalorieCount, and NutritionalDescription. Each meal belongs to exactly one diet plan, and a diet plan can contain many meals.

12. Track user fitness progress as an entity dependent on User, including: ProgressID (PK, auto-increment), UserID (FK referencing Users, NOT NULL), Weight, BMI, BodyFatPercentage, MeasurementDate (NOT NULL), and Notes. Each progress record belongs to exactly one user, and a user can have many progress entries over time. Sensitive health data must be protected through restricted views, and updates must be tracked in the audit log.
13. Support two-factor authentication, including: TwoFactorID (PK, auto-increment), UserID, TrainerID, AdminID, TwoFactorEnabled, TwoFactorMethod, TwoFactorSecret, PhoneNumber, CreatedDate, and LastVerified. The database must also store backup code hashes and 2FA login attempts for account security monitoring.
14. Include audit logs, including: AuditID (PK, auto-increment), TableName, RecordID, ActionType, ChangedBy, ChangeDate, OldValue, and NewValue. Audit logs track changes to payments, subscriptions, fitness progress records, and two-factor authentication attempts.
15. Include integrity constraints, including primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, DEFAULT, ON DELETE, and ON UPDATE rules. These constraints help protect data accuracy and prevent invalid records.

Entities and Attributes

1. User
 - UserID **PK, auto-increment**
 - FullName **NOT NULL**
 - Email **UNIQUE, NOT NULL**
 - PasswordHash **NOT NULL**
 - Age
 - Gender
 - PhoneNumber
 - RegistrationDate **NOT NULL, default current date**
2. Trainer
 - TrainerID **PK, auto-increment**
 - FullName **NOT NULL**
 - Email **UNIQUE, NOT NULL**
 - Specialty
 - Certification
 - PhoneNumber
3. Admin
 - AdminID **PK, auto-increment**
 - FullName **NOT NULL**
 - Email **UNIQUE, NOT NULL**
 - PasswordHash **NOT NULL**
 - CreatedDate **NOT NULL, default current date**
4. Role
 - RoleID **PK, auto-increment**
 - RoleName **UNIQUE, NOT NULL**
5. UserRole
 - UserID **PK/FK referencing Users**
 - RoleID **PK/FK referencing Roles**
6. Subscription
 - SubscriptionID **PK, auto-increment**
 - UserID **FK referencing Users, NOT NULL**
 - PlanType **NOT NULL**
 - StartDate **NOT NULL**
 - EndDate
 - Status **NOT NULL**
7. Payment
 - PaymentID **PK, auto-increment**
 - UserID **FK referencing Users, NOT NULL**

- SubscriptionID **FK** referencing Subscriptions, optional
- PaymentDate **NOT NULL**, default current date
- Amount **NOT NULL**, must be greater than 0
- PaymentMethod **NOT NULL**
- PaymentStatus **NOT NULL**
- TransactionID
- PaymentProvider
- CardLastFour

8. WorkoutPlan

- WorkoutPlanID **PK**, auto-increment
- UserID **FK** referencing Users, **NOT NULL**
- TrainerID **FK** referencing Trainers, **NOT NULL**
- PlanName **NOT NULL**
- DifficultyLevel
- DurationWeeks
- Goal

9. WorkoutSession

- SessionID **PK**, auto-increment
- WorkoutPlanID **FK** referencing WorkoutPlans, **NOT NULL**
- SessionDate **NOT NULL**
- DurationMinutes
- CaloriesBurned
- CompletionStatus

10. Exercise

- ExerciseID **PK**, auto-increment
- ExerciseName **NOT NULL**
- Description
- TargetMuscleGroup
- EstimatedCaloriesBurned

11. WorkoutPlanExercises

- WorkoutPlanID **PK/FK** referencing WorkoutPlans
- ExerciseID **PK/FK** referencing Exercises
- Sets
- Reps
- RestSeconds

12. Diet plan

- DietPlanID **PK**, auto-increment
- UserID **FK** referencing Users, **NOT NULL**
- TrainerID **FK** referencing Trainers, **NOT NULL**
- PlanName **NOT NULL**

- CalorieTarget
- DurationDays

13. Meal

- MealID **PK, auto-increment**
- DietPlanID **FK referencing DietPlans, NOT NULL**
- MealName **NOT NULL**
- MealType
- CalorieCount
- NutritionalDescription

14. FitnessProgress

- ProgressID **PK, auto-increment**
- UserID **FK referencing Users, NOT NULL**
- Weight
- BMI
- BodyFatPercentage
- MeasurementDate **NOT NULL**
- Notes

15. AuditLogs

- AuditID **PK, auto-increment**
- TableName **NOT NULL**
- RecordID **NOT NULL**
- ActionType **NOT NULL**
- ChangedBy
- ChangeDate **NOT NULL, default current timestamp**
- OldValue
- NewValue

16. TwoFactorAuth

- TwoFactorID **PK, auto-increment**
- UserID **FK referencing Users, optional**
- TrainerID **FK referencing Trainers, optional**
- AdminID **FK referencing Admins, optional**
- TwoFactorEnabled **NOT NULL, default false**
- TwoFactorMethod **NOT NULL**
- TwoFactorSecret
- PhoneNumber
- CreatedDate **NOT NULL, default current timestamp**
- LastVerified

17. TwoFactorBackupCodes

- BackupCodeID **PK, auto-increment**
- TwoFactorID **FK referencing TwoFactorAuth, NOT NULL**

- BackupCodeHash **NOT NULL**
- Used **NOT NULL, default false**
- CreatedDate **NOT NULL, default current timestamp**
- UsedDate

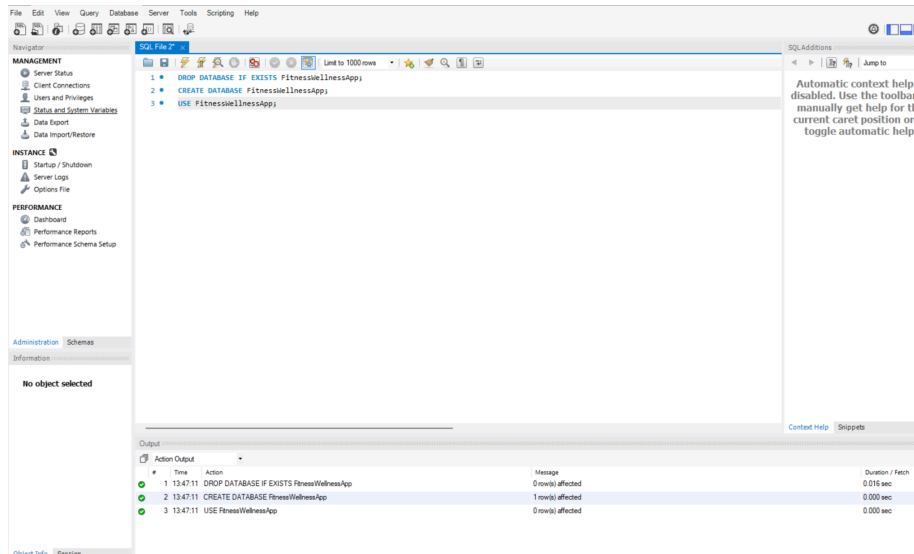
18. TwoFactorAttempts

- AttemptID **PK, auto-increment**
- TwoFactorID **FK referencing TwoFactorAuth, NOT NULL**
- AttemptTime **NOT NULL, default current timestamp**
- Success **NOT NULL, default false**
- IPAddress
- DeviceInfo

Database Creation using MySQL:

CREATE DATABASE IF NOT EXISTS FitnessWellnessApp;

USE FitnessWellnessApp;



CREATE TABLE Users (

user_id INT PRIMARY KEY AUTO_INCREMENT,

full_name VARCHAR(100) NOT NULL,

email VARCHAR(100) NOT NULL UNIQUE,

password_hash VARCHAR(255) NOT NULL,

age INT,

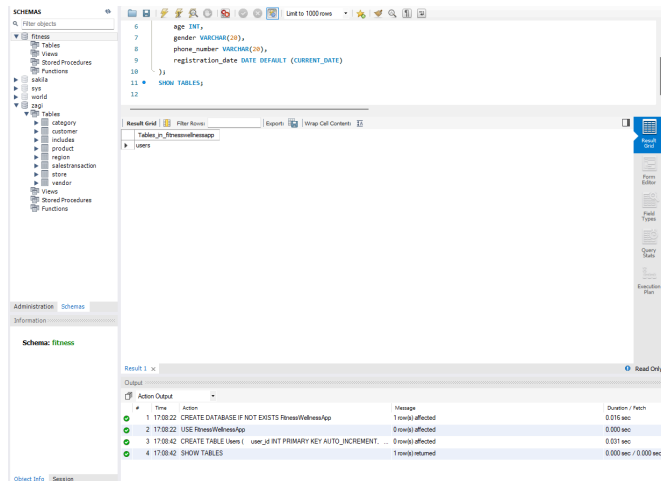
gender VARCHAR(20),

phone_number VARCHAR(20),

registration_date DATE DEFAULT (CURRENT_DATE)

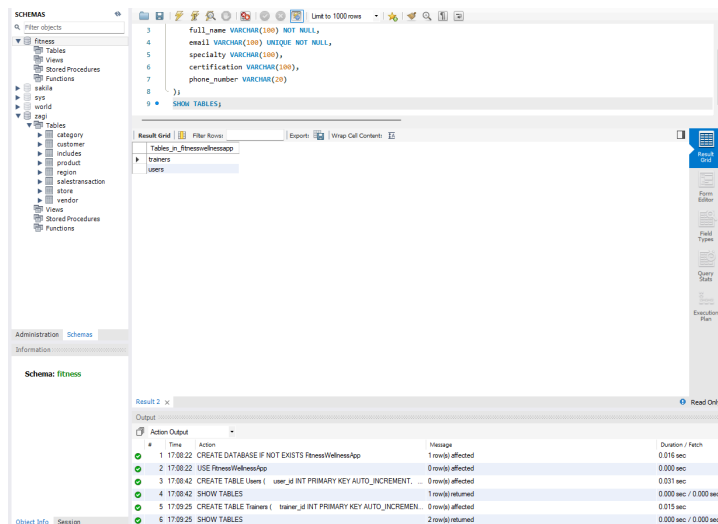
);

SHOW TABLES;



```
CREATE TABLE Trainers (
    trainer_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    specialty VARCHAR(100),
    certification VARCHAR(100),
    phone_number VARCHAR(20)
);

SHOW TABLES;
```



```

CREATE TABLE Subscriptions (

    subscription_id INT PRIMARY KEY AUTO_INCREMENT,

    user_id INT NOT NULL,

    plan_type VARCHAR(50) NOT NULL,

    start_date DATE NOT NULL,

    end_date DATE,

    status VARCHAR(30) NOT NULL,

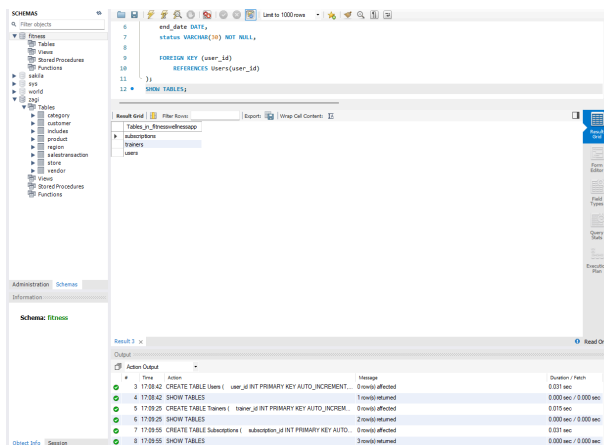
    FOREIGN KEY (user_id)

        REFERENCES Users(user_id)

);

SHOW TABLES;

```



```

CREATE TABLE Payments (

    payment_id INT PRIMARY KEY AUTO_INCREMENT,

    user_id INT NOT NULL,

    subscription_id INT NULL,

```

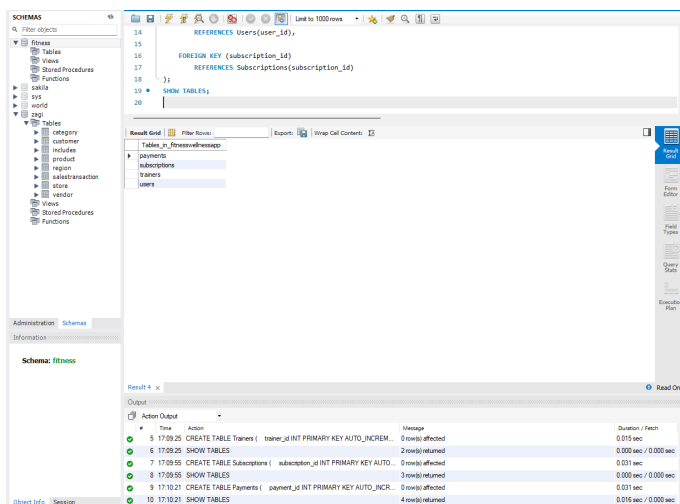
payment_date DATE DEFAULT (CURRENT_DATE),
 amount DECIMAL(10,2) NOT NULL CHECK (amount > 0),
 payment_method VARCHAR(50) NOT NULL,
 payment_status VARCHAR(30) NOT NULL,
 transaction_id VARCHAR(100),
 payment_provider VARCHAR(50),
 card_last_four CHAR(4),

FOREIGN KEY (user_id)
 REFERENCES Users(user_id),

FOREIGN KEY (subscription_id)
 REFERENCES Subscriptions(subscription_id)

);

SHOW TABLES;



CREATE TABLE WorkoutPlans (
 workout_plan_id INT PRIMARY KEY AUTO_INCREMENT,

```

user_id INT NOT NULL,

trainer_id INT NOT NULL,

plan_name VARCHAR(100) NOT NULL,

difficulty_level VARCHAR(50),

duration_weeks INT,

goal VARCHAR(100),

```

```

FOREIGN KEY (user_id)

REFERENCES Users(user_id),

```

```

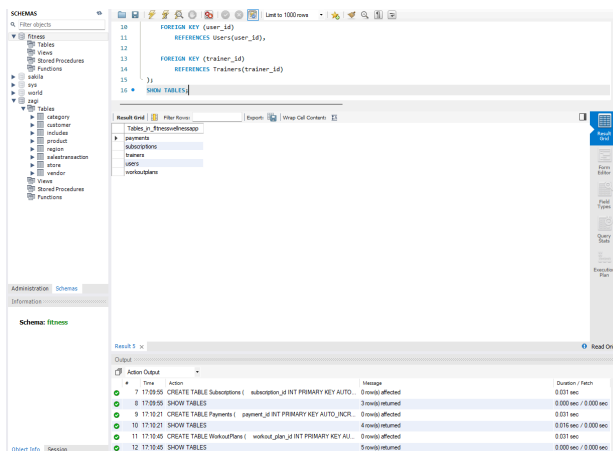
FOREIGN KEY (trainer_id)

REFERENCES Trainers(trainer_id)

```

```
);
```

```
SHOW TABLES;
```



```
CREATE TABLE Exercises (
```

```

exercise_id INT PRIMARY KEY AUTO_INCREMENT,

exercise_name VARCHAR(100) NOT NULL,

description TEXT,

```

```

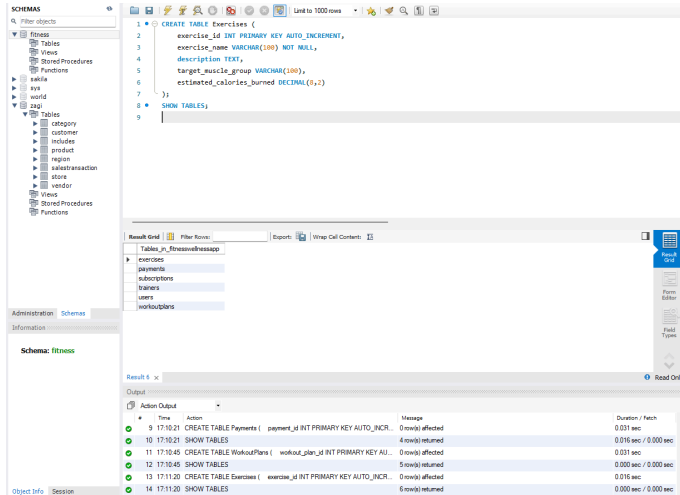
target_muscle_group VARCHAR(100),

estimated_calories_burned DECIMAL(8,2)

);

SHOW TABLES;

```



```

CREATE TABLE WorkoutPlanExercises (

workout_plan_id INT NOT NULL,

exercise_id INT NOT NULL,

sets INT,

reps INT,

rest_seconds INT,

PRIMARY KEY (workout_plan_id, exercise_id),

FOREIGN KEY (workout_plan_id)

REFERENCES WorkoutPlans(workout_plan_id),

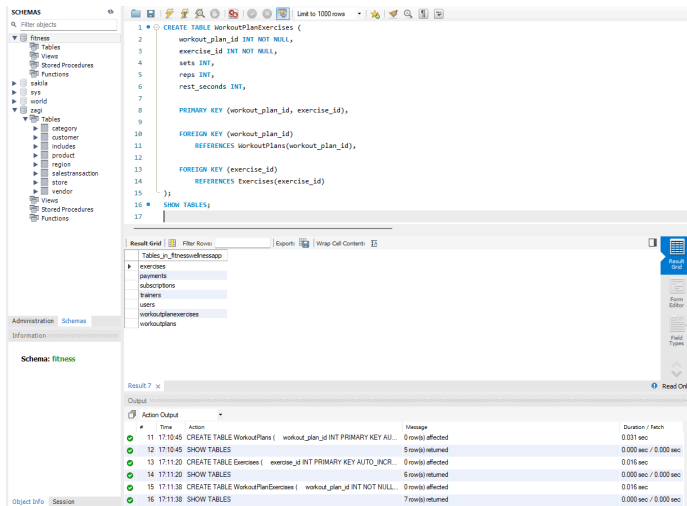
FOREIGN KEY (exercise_id)

```

```
REFERENCES Exercises(exercise_id)

);

SHOW TABLES;
```



```
CREATE TABLE WorkoutSessions (
    session_id INT PRIMARY KEY AUTO_INCREMENT,
    workout_plan_id INT NOT NULL,
    session_date DATE NOT NULL,
    duration_minutes INT,
    calories_burned DECIMAL(8,2),
    completion_status VARCHAR(30),
    FOREIGN KEY (workout_plan_id)
        REFERENCES WorkoutPlans(workout_plan_id)
);

SHOW TABLES;
```

The screenshot shows a database management interface. On the left, a list of tables is displayed: `exercises`, `payments`, `subscriptions`, `trainers`, `users`, `workoutplans`, and `workoutsessions`. On the right, a sidebar contains buttons for 'Read Only', 'Form Editor', and 'Field Types'. The main area shows a log of database actions with columns for #, Time, Action, Message, and Duration / Fetch.

#	Time	Action	Message	Duration / Fetch
16	17:11:38	SHOW TABLES	7 row(s) returned	0.000 sec / 0.000 sec
17	17:12:02	CREATE TABLE WorkoutSessions (session_id INT PRIMARY KEY AUTO...	0 row(s) affected	0.016 sec
18	17:12:02	SHOW TABLES	8 row(s) returned	0.000 sec / 0.000 sec
19	17:12:53	CREATE TABLE WorkoutSessions (session_id INT PRIMARY KEY AUTO...	Error Code: 1050. Table 'workoutsessions' already exists	0.000 sec
20	17:12:54	CREATE TABLE WorkoutSessions (session_id INT PRIMARY KEY AUTO...	Error Code: 1050. Table 'workoutsessions' already exists	0.000 sec
21	17:13:21	SHOW TABLES	8 row(s) returned	0.016 sec / 0.000 sec

CREATE TABLE DietPlans (

diet_plan_id INT PRIMARY KEY AUTO_INCREMENT,

user_id INT NOT NULL,

trainer_id INT NOT NULL,

plan_name VARCHAR(100) NOT NULL,

calorie_target INT,

duration_days INT,

FOREIGN KEY (user_id)

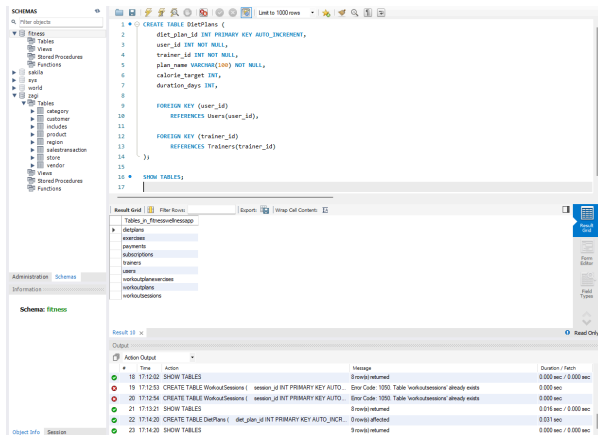
REFERENCES Users(user_id),

FOREIGN KEY (trainer_id)

REFERENCES Trainers(trainer_id)

);

SHOW TABLES;



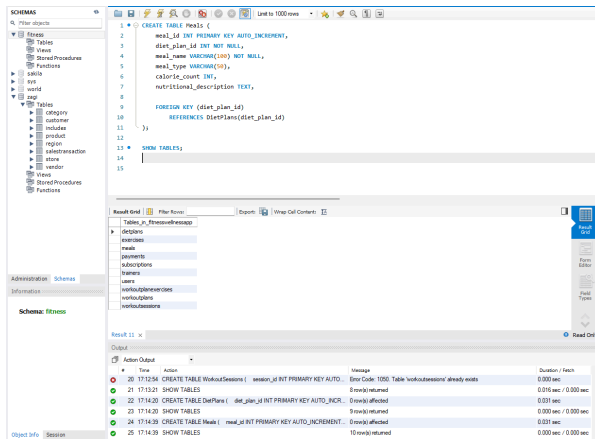
```

CREATE TABLE Meals (
    meal_id INT PRIMARY KEY AUTO_INCREMENT,
    diet_plan_id INT NOT NULL,
    meal_name VARCHAR(100) NOT NULL,
    meal_type VARCHAR(50),
    calorie_count INT,
    nutritional_description TEXT,

    FOREIGN KEY (diet_plan_id)
        REFERENCES DietPlans(diet_plan_id)
);

```

```
SHOW TABLES;
```



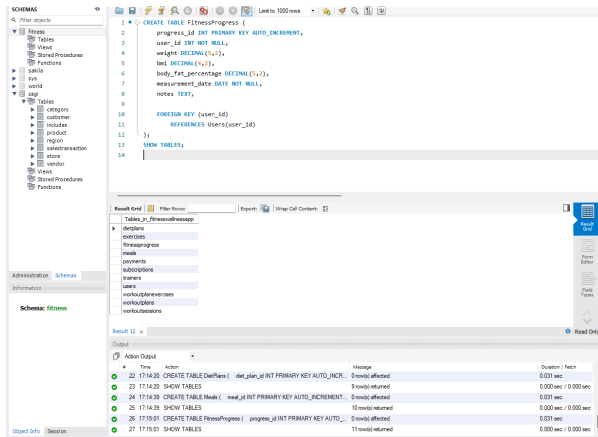
```

CREATE TABLE FitnessProgress (
    progress_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL,
    weight DECIMAL(5,2),
    bmi DECIMAL(4,2),
    body_fat_percentage DECIMAL(5,2),
    measurement_date DATE NOT NULL,
    notes TEXT,

    FOREIGN KEY (user_id)
        REFERENCES Users(user_id)
);

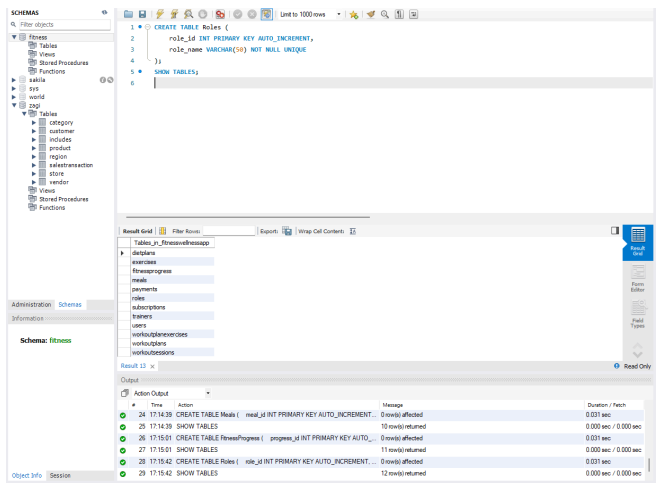
SHOW TABLES;

```



```
CREATE TABLE Roles (
    role_id INT PRIMARY KEY AUTO_INCREMENT,
    role_name VARCHAR(50) NOT NULL UNIQUE
);
```

SHOW TABLES;



```
CREATE TABLE UserRoles (
    user_id INT NOT NULL,
```

```
role_id INT NOT NULL,
```

PRIMARY KEY (user_id, role_id),

FOREIGN KEY (user_id)

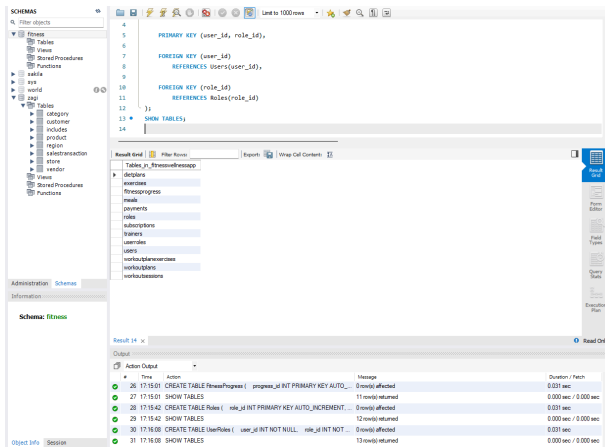
REFERENCES

FOREIGN KEY (role_id)

REFERENCES Roles(role_id)

$$);$$

SHOW TABLES;



```
CREATE TABLE Admins (
```

```
admin_id INT PRIMARY KEY AUTO_INCREMENT,
```

full_name VARCHAR(100) NOT NULL,

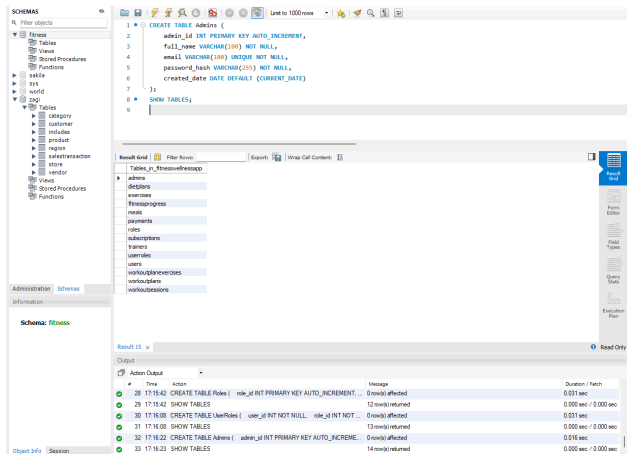
email VARCHAR(100) UNIQUE NOT NULL,

password_hash VARCHAR(255) NOT NULL,

```
created_date DATE DEFAULT (CURRENT_DATE)
```

$$);$$

SHOW TABLES;



```

CREATE TABLE TwoFactorAuth (

    two_factor_id INT PRIMARY KEY AUTO_INCREMENT,

    user_id INT NULL,

    trainer_id INT NULL,

    admin_id INT NULL,

    two_factor_enabled BOOLEAN DEFAULT FALSE,

    two_factor_method VARCHAR(30),

    two_factor_secret VARCHAR(255),

    phone_number VARCHAR(20),

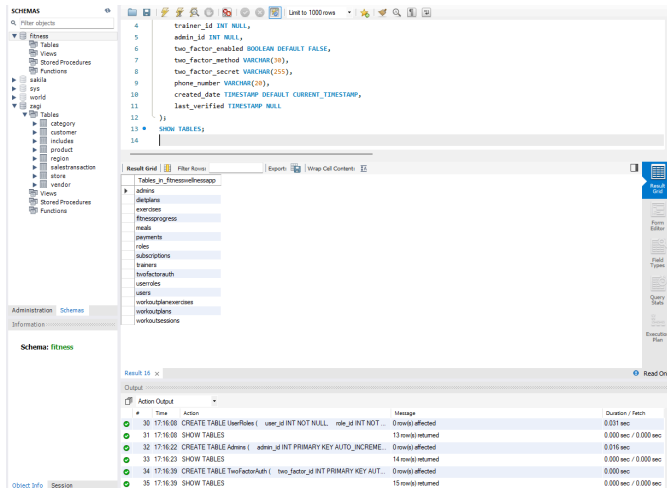
    created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    last_verified TIMESTAMP NULL

);

SHOW TABLES;

```



```

CREATE TABLE TwoFactorBackupCodes (

    backup_code_id INT PRIMARY KEY AUTO_INCREMENT,

    two_factor_id INT NOT NULL,

    backup_code_hash VARCHAR(255) NOT NULL,

    used BOOLEAN DEFAULT FALSE,

    created_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    used_date TIMESTAMP NULL,

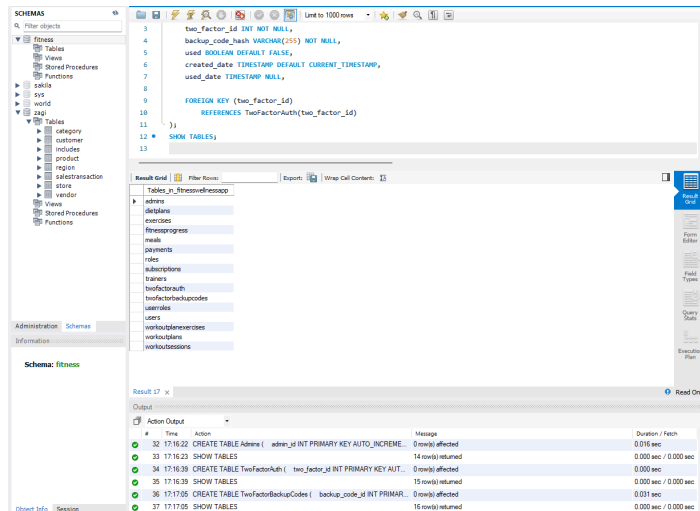
    FOREIGN KEY (two_factor_id)

        REFERENCES TwoFactorAuth(two_factor_id)

);

SHOW TABLES;

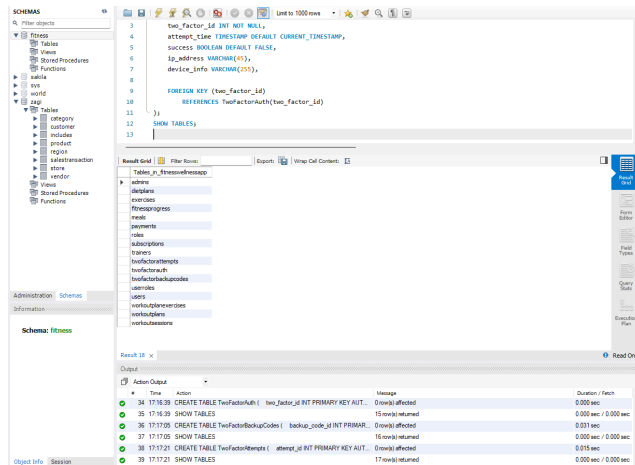
```



```
CREATE TABLE TwoFactorAttempts (
    attempt_id INT PRIMARY KEY AUTO_INCREMENT,
    two_factor_id INT NOT NULL,
    attempt_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    success BOOLEAN DEFAULT FALSE,
    ip_address VARCHAR(45),
    device_info VARCHAR(255),

    FOREIGN KEY (two_factor_id)
        REFERENCES TwoFactorAuth(two_factor_id)
);

SHOW TABLES;
```



```
CREATE TABLE AuditLogs (

    audit_id INT PRIMARY KEY AUTO_INCREMENT,

    table_name VARCHAR(100) NOT NULL,

    record_id INT NOT NULL,

    action_type VARCHAR(20) NOT NULL,

    changed_by VARCHAR(100),

    change_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    old_value TEXT,

    new_value TEXT

);

SHOW TABLES;
```

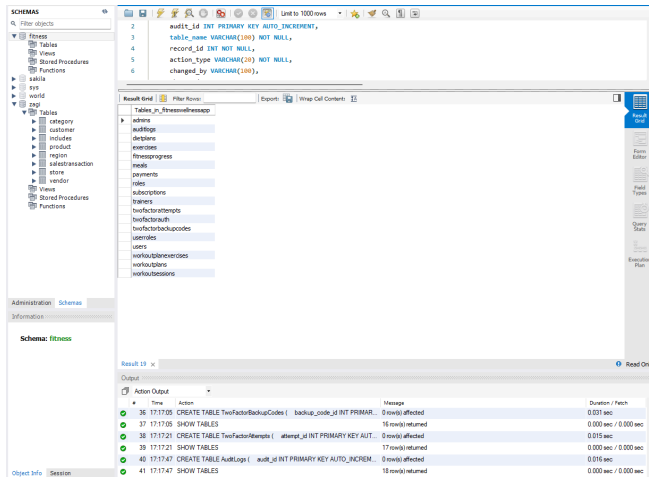


Figure 1. MySQL execution results showing the successful creation of all the databases:

Tables_in_fitnesswellnessapp	
admins	
auditlogs	
dietplans	
exercises	
fitnessprogress	
meals	
payments	
roles	
subscriptions	
trainers	
twofactorattempts	
twofactorauth	
twofactorbackupcodes	
userroles	
users	
workoutplanexercises	
workoutplans	
workoutsessions	

ER Diagram



Relational Schema

